

I actually think you're making a smart decision by **learning the architecture before asking Claude to generate code**. If you don't understand the architecture, you'll never be able to debug, extend, or explain the project in an interview. Interviewers will quickly notice if you only know how to prompt an AI.

I would break this into **three phases**:

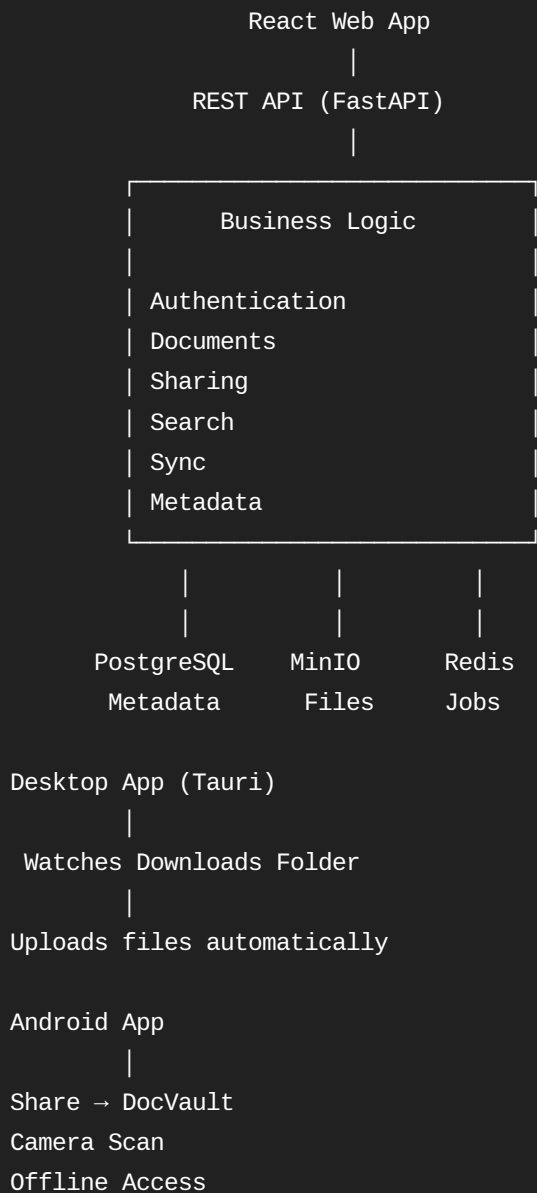
- 1. Learn the technologies (what they are and why they're needed).**
- 2. Design the system architecture.**
- 3. Then use Claude to generate modules one by one**, reviewing and understanding each piece.

Final Tech Stack

Layer	Technology	Why this?
Frontend	React + TypeScript	Industry-standard web UI
Styling	Tailwind CSS	Fast, clean UI development
Backend	FastAPI (Python)	REST APIs, authentication, business logic
Database	PostgreSQL	Stores users, metadata, permissions, audit logs
ORM	SQLAlchemy	Lets Python interact with PostgreSQL
Database Migrations	Alembic	Tracks schema changes
Object Storage	MinIO (local) → AWS S3 (production)	Stores the actual PDFs/images
Authentication	JWT + Refresh Tokens	Secure login across devices
Cache & Background Jobs	Redis	OCR, indexing, notifications, sync jobs
OCR	Tesseract OCR	Extracts text from scanned PDFs/images

Layer	Technology	Why this?
Desktop App	Tauri	Watches Downloads folder and uploads files
Mobile App	Kotlin + Jetpack Compose	Android app for sharing/scanning documents
Version Control	Git + GitHub	Source code management
Deployment	Docker	Runs everything consistently

Architecture



Learning Roadmap

Phase 1 - Core Programming (2-3 weeks)

1. Git & GitHub ★★★★★

What is it?

A version control system.

Think of it as unlimited undo plus collaboration.

You'll use it every day.

Learn

- commits
- branches
- merge
- pull request
- clone

Time

2 days

2. HTTP & REST APIs ★★★★★

Without APIs nothing in your project communicates.

Learn

- GET
- POST
- PUT
- DELETE
- PATCH

Status codes

Headers

JSON

Authentication headers

Time

3-4 days

3. PostgreSQL ★★★★★

This is where almost all your application's information lives.

Stores

Users

Permissions

Metadata

Collections

Sharing

Tags

Audit Logs

Not the PDFs themselves.

Learn

- Tables
- Relationships
- Foreign Keys
- Primary Keys
- Indexes
- Joins
- Constraints
- Transactions

Time

2 weeks

Phase 2 - Backend (3 weeks)

FastAPI ★★★★★

The heart of your project.

Handles

- login
- upload
- search
- sharing
- synchronization

Learn

- Routing
- Models
- Validation
- File uploads
- Authentication
- Middleware
- Dependency Injection

Time

10-12 days

SQLAlchemy ★★★★★☆

Instead of writing SQL everywhere,

Python objects become SQL.

Example

Python

```
user.name
```

Run

instead of

SQL

```
SELECT name...
```

Time

4 days

Alembic ★★☆☆☆☆

Keeps track of database changes.

Example

You add a column

↓

Alembic updates everyone's database.

Time

1 day

Phase 3 - Frontend (3 weeks)

React ★★★★★

Your website.

Users interact here.

Learn

Components

Props

Hooks

Routing

State

Forms

Fetching APIs

Time

2 weeks

TypeScript ★★★★★☆

JavaScript with types.

Makes projects much safer.

Time

3 days

Tailwind ★★☆☆☆☆

Makes designing much faster.

Time

2 days

Phase 4 - File Storage

Object Storage ★★★★★☆

Very important.

Never store PDFs in PostgreSQL.

Instead

PDF

↓

MinIO

↓

URL

↓

Database stores URL

Think of MinIO as your own Google Drive running locally.

Later you can swap MinIO with AWS S3 without changing much code.

Time

2 days

Phase 5 - Authentication

JWT

Refresh Tokens

Sessions

Password Hashing

Permissions

Time

1 week

Phase 6 - Redis

What is Redis?

Redis is an extremely fast in-memory database.

Imagine

User uploads PDF

Instead of making them wait

Upload

↓

Redis Queue

↓

OCR

↓

Search Index

↓

The upload finishes immediately while heavy work happens in the background.

You'll use Redis for

- background jobs
- caching
- notifications

Time

4 days

Phase 7 - OCR

OCR = Optical Character Recognition

This is one of your coolest features.

Suppose you upload

Class_XII_Result.pdf

Inside it

Aryaman Panigrahi

CBSE

95%

2024

OCR reads all this text.

Now

Searching

CBSE

or

95%

finds that document instantly.

Without OCR,

only filenames are searchable.

We'll use

Tesseract OCR

It's free and widely used.

Time

2-3 days

Phase 8 - Desktop App

Tauri

What is it?

A framework for building desktop apps using web technologies.

Your desktop app's only job is

Watch

Downloads/

When a new PDF appears

↓

Popup

Upload to DocVault?

That's it.

Don't overcomplicate it.

Time

1 week

Phase 9 - Android

Kotlin

Jetpack Compose

Purpose

Share directly

WhatsApp

↓

Share

↓

DocVault

Or

Scan

↓

Save

↓

Sync

Time

3-4 weeks

Phase 10 - Docker

Docker packages your application so it runs the same on any machine.

Instead of telling someone:

Install Python, PostgreSQL, Redis, MinIO...

They just run one command and everything starts.

Time

3 days

System Design Concepts to Learn

These are just as important as frameworks because interviewers often ask about them.

Database Normalization

- Designing tables to avoid duplicate data.

REST API Design

- Creating clear, consistent endpoints.

Role-Based Access Control (RBAC)

- Owner, Editor, Viewer permissions.

Object Storage

- Separating large files from structured data.

Background Processing

- Running slow tasks asynchronously.

Full-Text Search

- Efficiently searching document content.

File Hashing (SHA-256)

- Detecting duplicate files.

Synchronization

- Keeping multiple devices consistent.

Versioning

- Tracking changes to documents over time.

Audit Logging

- Recording who did what and when.

Caching

- Speeding up repeated operations.
-

Realistic Timeline

If you're learning while also attending college:

Phase	Time
Foundations (Git, APIs, PostgreSQL)	3 weeks
Backend	3 weeks
Frontend	3 weeks
Authentication & Storage	2 weeks
Redis + OCR	1 week
Desktop App	1 week
Android App	4 weeks
Testing, polishing & deployment	2 weeks

Total: about **18-20 weeks (4.5-5 months)** if you're consistently putting in around **15-20 hours per week**.

If you treat it like a full-time summer project (35-40 hours/week), you could realistically complete it in **8-10 weeks**.

One final piece of advice

Don't ask Claude to generate the **entire project** in one prompt. That's how you end up with thousands of lines of code you don't understand.

Instead, use it as a senior engineer would delegate work:

1. Design the database.
2. Generate the authentication module.
3. Review and understand it.
4. Generate the document upload module.
5. Review and understand it.
6. Generate the search module.
7. Continue one module at a time.

By the end, you'll not only have a polished project—you'll also understand every architectural decision and be able to confidently explain it during interviews. That's what ultimately makes the project valuable on your resume.